ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

BÀI GIẢNG
LẬP TRÌNH MẠNG

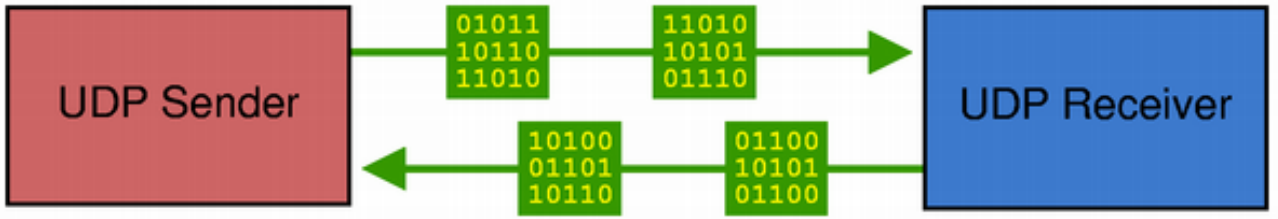
PGS.TS. Huỳnh Công Pháp; Nguyễn Anh Tuấn;
Lê Thành Công; Nguyễn Thanh Cẩm, Lê Tân
Khoa Khoa học máy tính

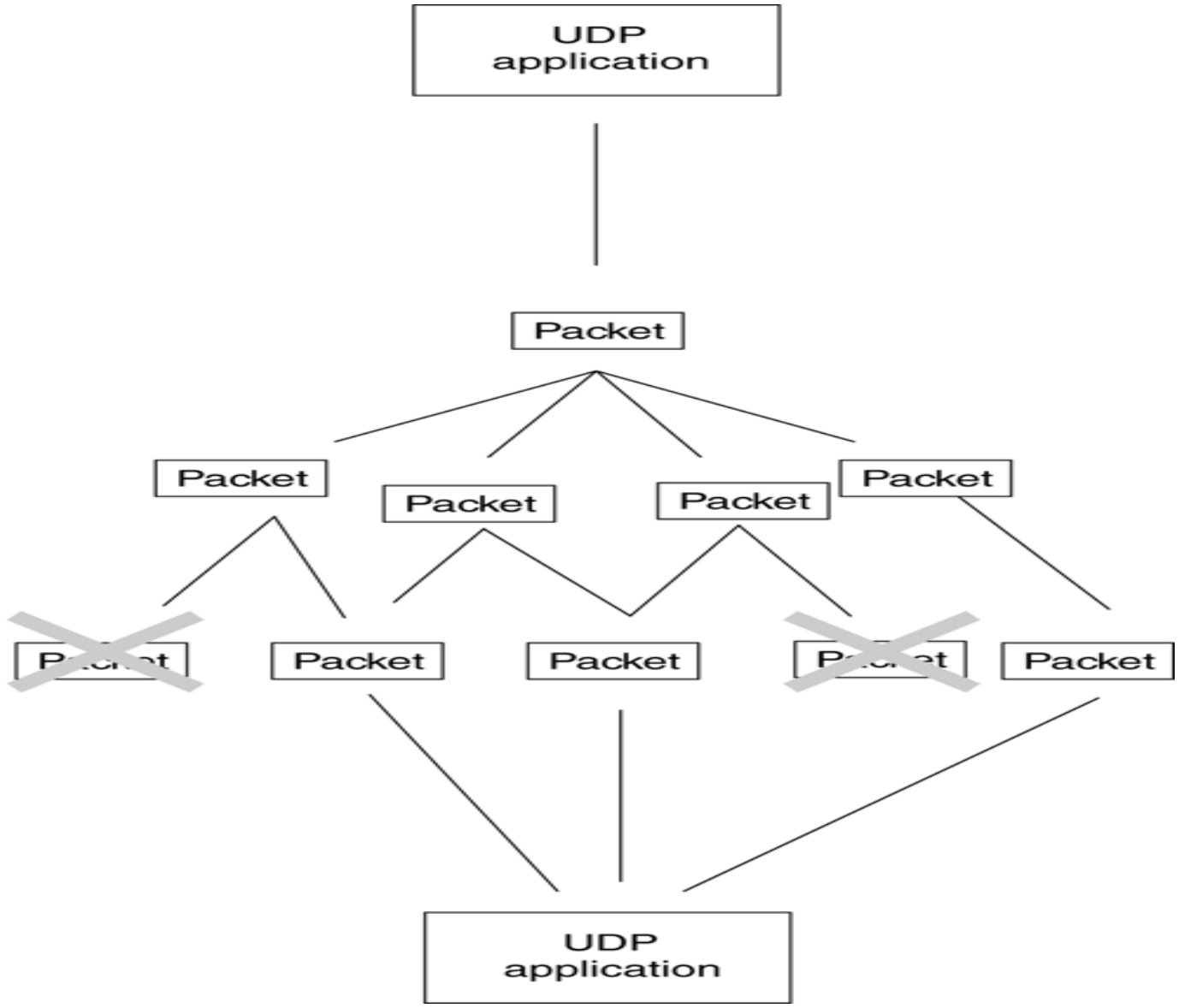
Bài 5. Lập trình với giao thức UDP

(User Datagram Protocol)

Giao thức UDP

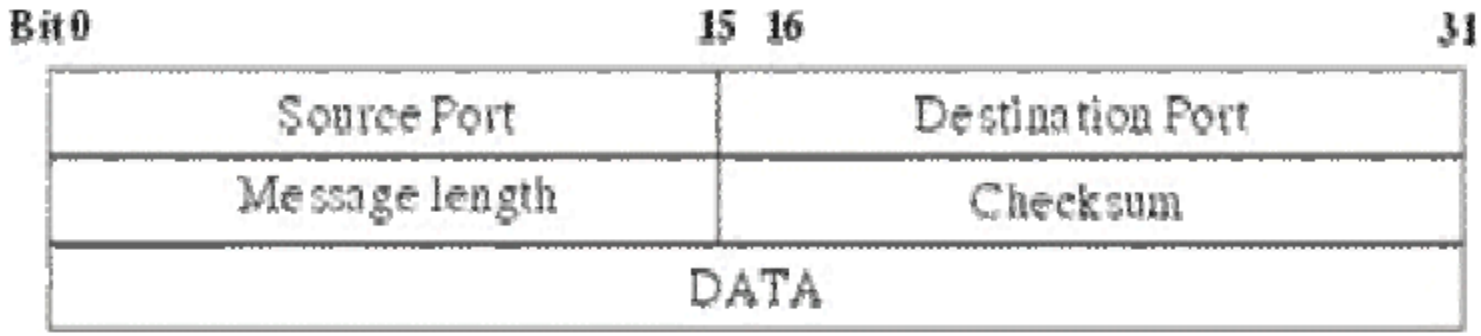
- UDP (User Datagram Protocol) là giao thức phi kết nối
  - Không thiết lập kết nối trước khi truyền (connectionless)
  - Các gói dữ liệu được gửi độc lập (gọi là datagrams)
  - Không bảo đảm sự toàn vẹn dữ liệu và trật tự.
  - UDP không cung cấp cơ chế báo nhận (acknowledgment),
- Cơ chế hoạt động tương tự như gửi một lá thư thông qua dịch vụ bưu điện




UDP application
Packet
Packet
Packet
Packet
Packet
Packet
Packet
Packet
Packet
UDP application

Khuôn dạng UDP datagram

- UDP datagram có các tham số đơn giản hơn nhiều so với TCP segment.



- UDP cũng cung cấp cơ chế gán và quản lý các số hiệu cổng (port number) để định danh duy nhất cho các ứng dụng chạy trên một trạm của mạng.
- Do ít chức năng phức tạp nên UDP thường có xu thế hoạt động nhanh hơn so với TCP.
- UDP thường dùng cho các ứng dụng không đòi hỏi độ tin cậy cao trong khi truyền.

Các bước tạo ứng dụng UDP Socket

- ứng dụng UDP socket không thiết lập kết nối
  - Client
    - socket, tạo một điểm cuối truyền thông phía client
    - sendto and recvfrom, truyền dữ liệu
  - Server
    - socket, Tạo một điểm cuối truyền thông phía server
    - bind, gắn một cổng đến kết nối
    - sendto and recvfrom, truyền dữ liệu



- server : socket → bind → recvfrom → sendto → close

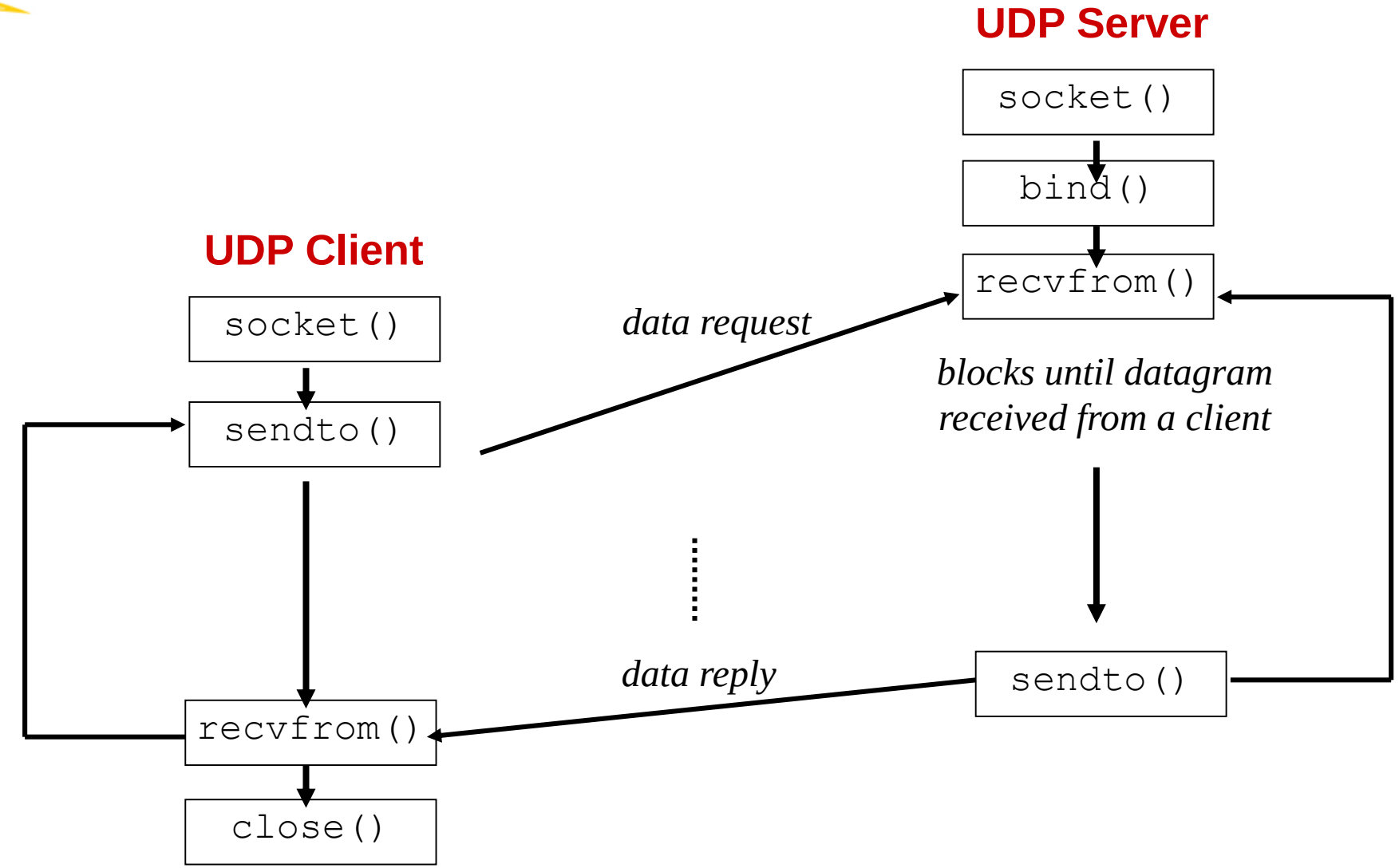


- client : socket → sendto → recvfrom → close

Lưu ý

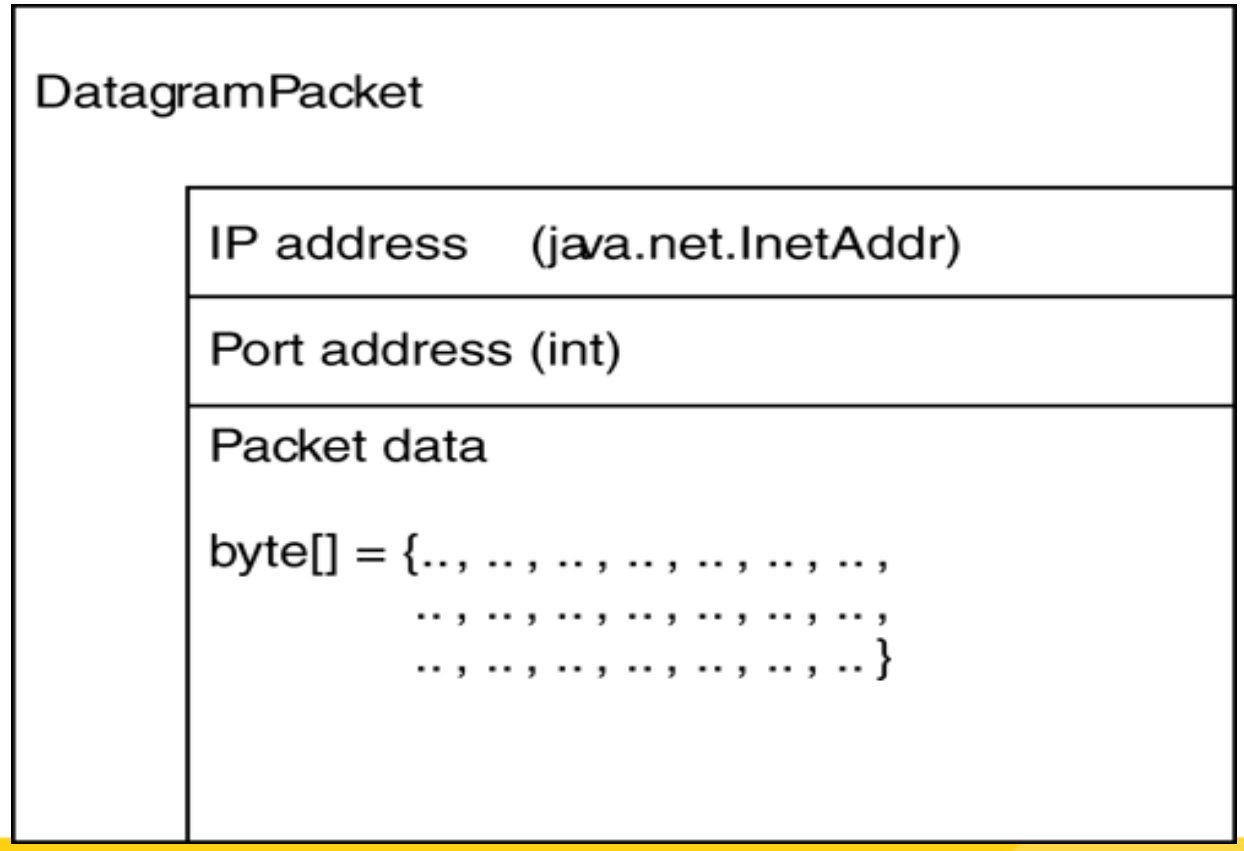
- Client không thiết lập kết nối đến server
  - Kết nối không cần thiết
- Server không chấp nhận kết nối
  - chờ và lắng nghe không cần thiết
  - chấp nhận kết nối không tồn tại

Các bước tạo ứng dụng UDP Socket



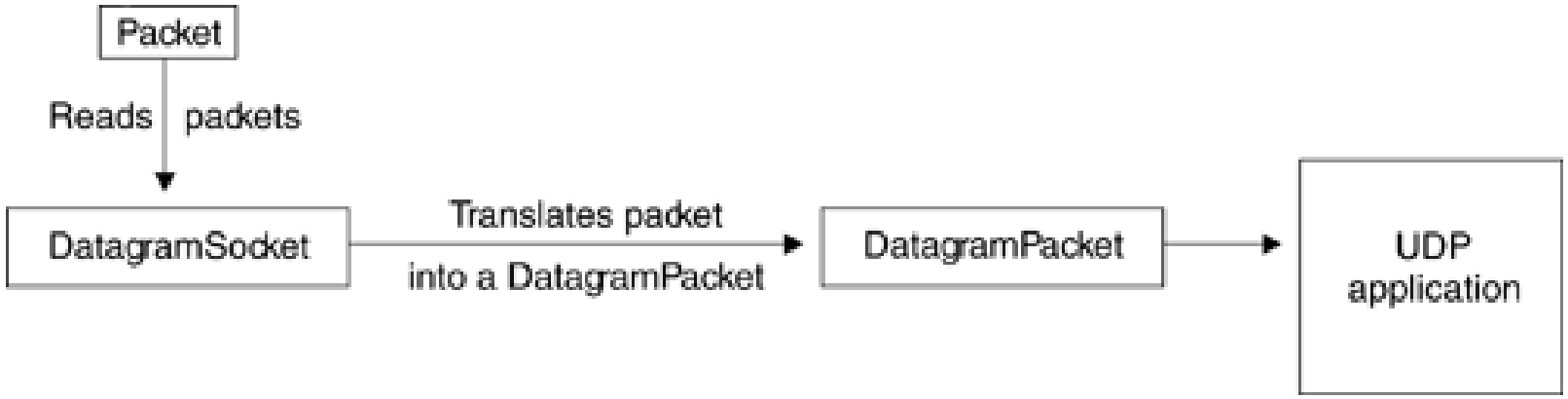
Tạo ứng dụng UDP socket với Java

- Sử dụng lớp DatagramPacket biểu diễn một packet dữ liệu truyền bằng giao thức UDP.
- Packets chứa dãy các bytes, và các thông tin địa chỉ như IP và số cổng.



Tạo ứng dụng UDP socket với Java

- Tạo DatagramSocket
  - Một DatagramSocket có thể dùng để gửi và nhận các packets dữ liệu. Mỗi DatagramSocket gắn đến một cổng, được dùng để đánh địa chỉ cho packet.
- Lắng nghe UDP Packets
  - Trước khi một ứng dụng có thể đọc được các UDP packets được gửi từ máy khác, nó phải gắn một socket đến một cổng UDP sử dụng DatagramSocket và tạo một DatagramPacket mà sẽ dùng như một bộ chứa dữ liệu UDP packet gửi đến.



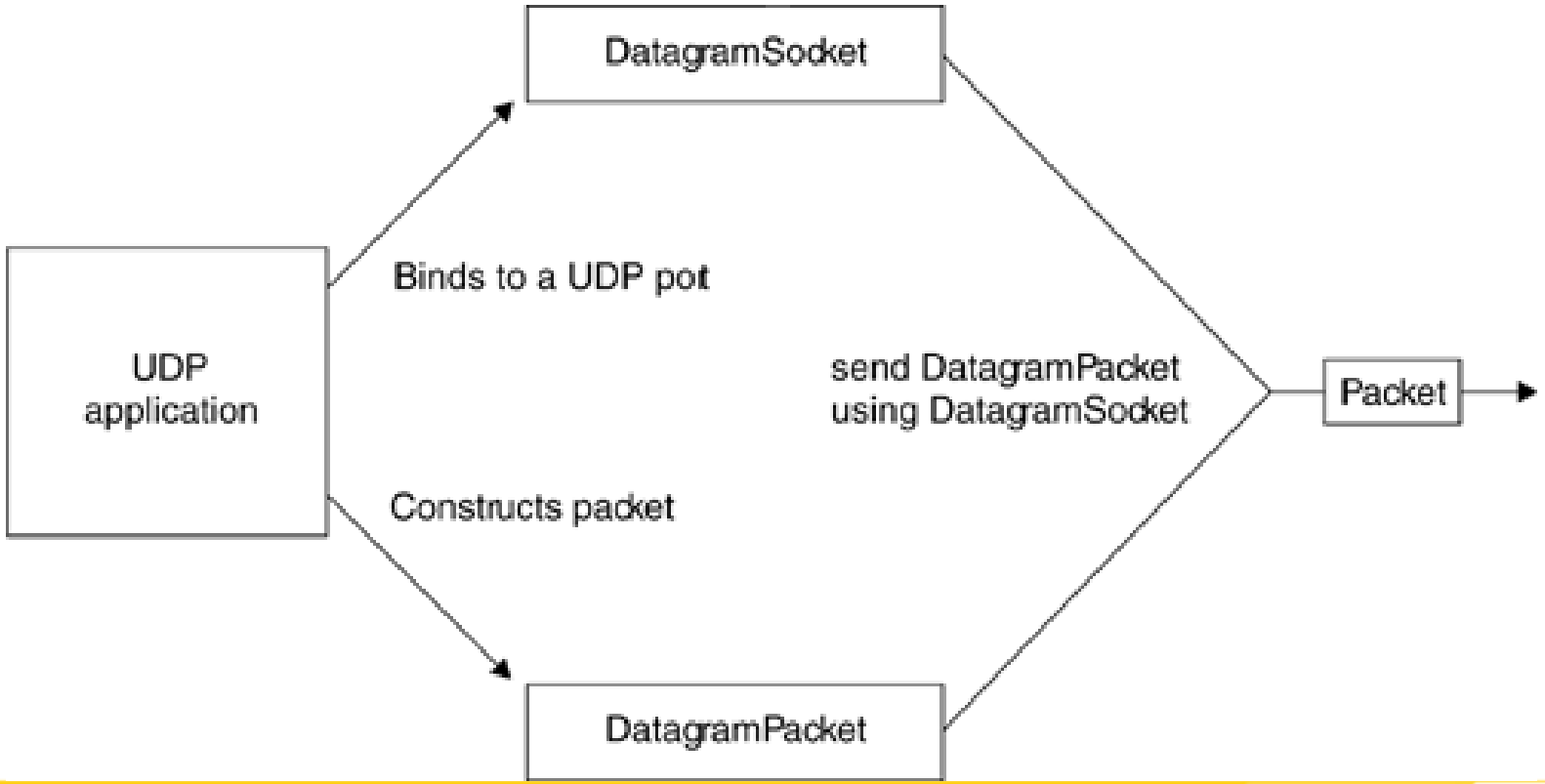
Tạo ứng dụng UDP socket với Java

- Đoạn mã Java nhận UDP packet

```
DatagramPacket packet = new DatagramPacket (new byte[256], 256);
DatagramSocket socket = new DatagramSocket(2000);
boolean finished = false;
while (!finished )
{
   socket.receive (packet);
   // process the packet
}
socket.close();
```

Gửi UDP packets

- DatagramSocket cũng dùng để gửi các packets dữ liệu
- Khi gửi một packet, ứng dụng phải tạo một DatagramPacket, thiết lập địa chỉ và thông tin cổng, và viết dữ liệu muốn gửi dưới dạng dãy các bytes



Đoạn mã Java gửi UDP packets

```
DatagramSocket socket = new DatagramSocket(1500);
DatagramPacket packet = new DatagramPacket (“hello”.getBytes());
packet.setAddress ( InetAddress.getByName ( “localhost”) );
packet.setPort ( 2000 );
boolean finished = false;
while (!finished )
{
   // Write data to packet buffer
   .........
   socket.send (packet);
   // Do something else, like read other packets, or check to
   // see if no more packets to send
   .........
}
socket.close();
```

Ví dụ chương trình Java: UDP Server

```
public class UDPServer {
    public static void main(String args[]) throws Exception {
        DatagramSocket sSocket = new DatagramSocket(98);
        DatagramPacket packet;
        byte[] data;
        Scanner sc = new Scanner(System.in);
        while(true){
            data = new byte[1024];
            packet = new DatagramPacket(data, data.length);
            sSocket.receive(packet);
            System.out.println("Received: " + (new String(packet.getData())));
            InetAddress addr = packet.getAddress();
            int port = packet.getPort();
            System.out.print("To Client: ");
            data = (sc.nextLine()).getBytes();
            packet = new DatagramPacket(data, data.length, addr, port);
            sSocket.send(packet);
        }
    }
}
```

Ví dụ chương trình Java: UDP Client

```
public class UDPClient {
    public static void main(String args[]) throws Exception {
        DatagramSocket cSocket = new DatagramSocket();//97
        DatagramPacket packet;
        byte[] data;
        InetAddress addr = InetAddress.getByName("localhost");
        Scanner sc = new Scanner(System.in);
        while(true){
            System.out.print("To Server: ");
            data = (sc.nextLine()).getBytes();
            packet = new DatagramPacket(data, data.length, addr, 98);
            cSocket.send(packet);
            data = new byte[1024];
            packet = new DatagramPacket(data, data.length);
            cSocket.receive(packet);
            System.out.println("Received: " + (new String(packet.getData())));
        }
    }
}
```

Lớp InetAddress

- Các thiết bị được kết nối với mạng LAN có địa chỉ vật lý duy nhất, giúp cho các máy khác trên mạng trong việc truyền các gói tin đến đúng vị trí. Tuy nhiên, địa chỉ này chỉ có ích trong mạng LAN.
- Một máy không thể xác định được vị trí trên Internet bằng cách sử dụng các địa chỉ vật lý, vì các địa chỉ vật lý không chỉ ra vị trí của máy.
- Hơn nữa, các máy thường di chuyển từ vị trí này sang vị trí khác, trong trường hợp máy xách tay hoặc máy palm chẳng hạn.

Lớp InetAddress

- Người lập trình mạng không quan tâm đến từng chi tiết dữ liệu được định tuyến như thế nào trong một mạng LAN.
- Java cũng không cung cấp khả năng truy xuất tới các giao thức tầng liên kết dữ liệu mức thấp được sử dụng bởi LAN.
- Thay vào đó, Java hỗ trợ giao thức TCP/IP, giao thức này có nhiệu vụ liên kết các mạng với nhau.

Lớp InetAddress

- Các thiết bị có một kết nối Internet trực tiếp được cung cấp một định danh duy nhất được gọi là địa chỉ IP, có thể là tĩnh hoặc động.
- Các địa chỉ IP được cấp phát động thường được sử dụng khi nhiều thiết bị cần truy cập Internet trong khoảng thời gian nhất định
- Địa chỉ này được sử dụng bởi giao thức IP để định tuyến các datagram tới đúng vị trí.

Lớp InetAddress

- Lớp java.net.InetAddress biểu diễn một địa chỉ Internet.
- Bao gồm hai trường thông tin: hostName (một đối tượng kiểu String) và address (một số kiểu int).
- Các trường này không phải là trường public, vì thế không thể truy xất chúng trực tiếp.
- Lớp này được sử dụng bởi hầu hết các lớp mạng, bao gồm Socket, ServerSocket, URL, DatagramSocket, DatagramPacket,…

Tạo các đối tượng InetAddress

- Lớp InetAddress được sử dụng để biểu diễn các địa chỉ IP trong một ứng dụng mạng sử dụng Java.
- Không giống với các lớp khác, không có các constructor cho lớp InetAddress.
- Tuy nhiên, lớp InetAddress có ba phương thức tĩnh trả về các đối tượng InetAddress

Các phương thức lớp InetAddress

- public static InetAddress InetAddress.getByName(String hostname)
- public static InetAddress[] InetAddress.getAllByName(String hostname)
- public static InetAddress InetAddress.getLocalHost()
- Tất cả các phương thức này đều thực hiện kết nối tới server DNS cục bộ để biết được các thông tin trong đối tượng InetAddress.

Các phương thức lớp InetAddress

- Xét phương thức đầu tiên. Phương thức này nhận tên của hostname làm tham số và trả về đối tượng kiểu InetAddress. Ví dụ:

```
try{
        InetAddress dc
        =InetAddress.getByName(“www.microsoft.com”);
        System.out.println(dc);
}
catch(UnknownHostException e) {
        System.err.println(e);
}
```

Các trường của đối tượng InetAddress

- Chỉ các lớp trong gói java.net có quyền truy xuất tới các trường của lớp InetAddress bằng cách gọi phương thức getHostName, getAddress() và getHostAddress().
  - public String getHostName(): trả về chuỗi biểu diễn hostname của đối tượng InetAddress. Nếu máy không có hostname thì trả về địa chỉ IP dưới dạng chuỗi.
  - public byte[] getAddress(): trả về một địa chỉ IP dưới dạng một mảng các byte.
  - public String getHostAddress(): trả về một địa chỉ IP dưới dạng chuỗi.

Các trường của đối tượng InetAddress

- Một số địa chỉ IP có các ý nghĩa đặc biệt. Ví dụ, 127.0.0.1 là địa chỉ loopback (localhost). Các địa chỉ IPv4 trong khoảng 224.0.0.0 tới 239.255.255.255 là các địa chỉ multicast.
- Java 1.5 thêm vào hai phương thức cho lớp InetAddress kiểm tra một nút cụ thể có đến được hay không từ nút hiện hành: kiểm tra xem một liên kết mạng đã được thiết lập hay chưa.
- Các liên kết có thể bị phong tỏa vì firewall, các server ủy quyền, các router hoạt động sai chức năng, dây cáp bị đứt, hoặc host ở xa không bật…

Các trường của đối tượng InetAddress

- public boolean isReachable(int timeout) throws IOException
- public boolean isReachable(NetworkInterface interface, int ttl, int timeout) throws IOException
- Các phương thức này cố gắng kết nối trên cổng echo trên host ở xa để tìm xem nó có thể đến được hay không. Nếu host đáp ứng trong khoảng thời gian timeout mili giây, các phương thức này trả về giá trị true, ngược lại nó trả về giá trị false.

Bài tập lập trình UDP Socket

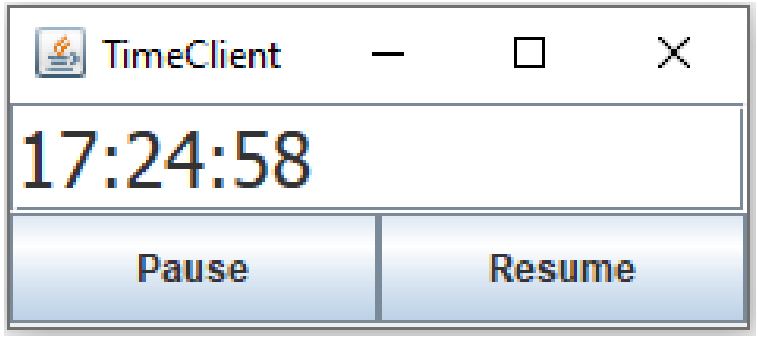
- Xây dựng chương trình ExchangeRate
  - ExchangeRateServer
    - chạy trên máy chủ và đón nhận dữ liệu do máy khách gửi đến cổng 2345. Khi nhận được yêu cầu, máy chủ sẽ gửi trả các thông báo về tỉ giá kèm theo ngày giờ mới nhất về cho máy khách. Có thể dùng các hàm random để lấy giá trị ngẫu nhiên của 3 thị trường là Tokyo, Newyork, và Hồng kông
  - ExchangeRateTable
    - Chạy trên máy khách, nó chịu trách nhiệm mỗi giây gởi yêu cầu đến máy chủ để cập nhật thông tin về tỉ giá của thị trường chứng khoán.

Bài tập lập trình UDP Socket

- Hãy tạo chương trình Mail Server.
  - Mỗi khi người dùng tạo account mới tại các máy client, thì chương trình server sẽ tạo một thư mục tương ứng ở máy server, đồng thời tạo một file có tên new_email.txt trong thư mục đó có nội dung như sau :
  - “Thank you for using this service. we hope that you will feel comfortabl.....”
  - Mỗi khi người dùng gửi một email, chương trình server nhận được email đó, xác định gửi đến account nào, tại thư mục của account đó chương trình server sẽ tạo một file có nội dung là nội dung email đó.
  - Mỗi khi người dùng ở máy client đăng nhập vào một account, thì chương trình server phải mở thư mục tương ứng với account đó ở server và gửi tất cả các tên file của thư mục đó về client

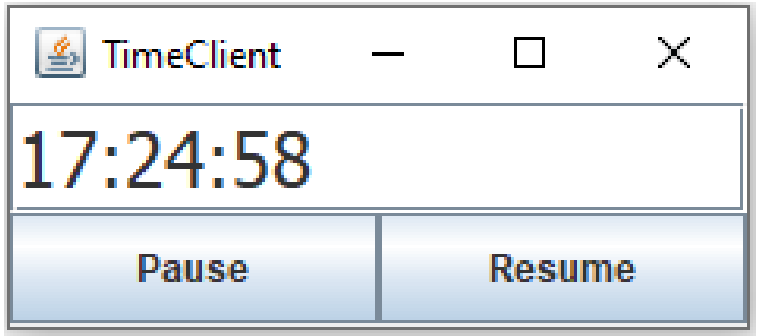
Bài tập lập trình UDP Socket

- Viết chương trình TimeUDPServer/TimeUDPClient
  - Server gởi time cho Client hiển thị sau mỗi giây
  - Thêm các nút Pause và Resume, Khi bấm các nút này thì Client gởi thông điệp phù hợp để Server tạm dừng/khôi phục việc gởi time cho Client đó.



Hướng dẫn

- Cách 1
  - Client gởi yêu cầu cho Server sau mỗi giây
  - Server nhận yêu cầu thì sẽ gởi dữ liệu cho Client hiển thị.
  - Bấm Pause: Client dừng gởi yêu cầu cho Server
  - Bấm Resume: Client tiếp tục gởi yêu cầu cho Server



Hướng dẫn

- Cách 2
  - Server sử dụng List<Thread>. Vòng lặp vô tận để đọc tín hiệu T
  - T = “login”: Tạo ra một Thread để đáp ứng cho Client đó, add Thread vô trong List
  - T = “logout”: Server tìm (trong List) Thread tương ứng, huỷ Thread này và remove ra khỏi List
  - T = “pause”: Server tìm Thread tương ứng, dừng Thread này
  - T = “resume”: Server tìm Thread tương ứng, cho chạy lại

Hướng dẫn

- Cách 3
  - Server sử dụng vòng lặp vô tận để đọc tín hiệu “login” trên DatagramSocket s;
  - Khi nhận tín hiệu “login” từ Client: Tạo ra một DatagramSocket t để giao tiếp với Client đó, và 2 Thread
    - Thread1: Sử dụng t để gởi dữ liệu cho Client
    - Thread2: Sử dụng t để đọc tín hiệu (logout, pause, resume) từ Client và điều khiển Thread1

Thank your listening